Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

A Liguagem SQL (continuação)



Sumário

1 Manipulação de Dados (DML)

2 Interrogação (Querying)

Bibliografia:

- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapter 18)**

- Belo, O., “Bases de Dados Relacionais: Implementação com MySQL”, FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5. **(Chapter 2)**

A Linguagem SQL

Consultas de Dados

- Após o povoamento inicial de uma base de dados, surge a necessidade de **explorar e analisar a informação armazenada**
- As consultas permitem **aceder, filtrar e organizar dados** de forma eficiente
- Em SQL, essa funcionalidade é garantida pela instrução **SELECT**
- A instrução **SELECT** é uma das mais importantes da linguagem, pela sua **flexibilidade e poder de análise**

A Linguagem SQL

SELECT

A sequência de processamento de uma instrução SELECT é:

- SELECT – especifica que colunas devem aparecer na saída
- FROM – especifica a tabela ou tabelas a serem utilizadas
- WHERE – filtra as linhas sujeitas a alguma condição
- GROUP BY forma grupos de linhas com o(s) mesmo(s) valor(es) da(s) coluna(s)
- HAVING – filtros de grupos sujeitos a alguma condição
- ORDER BY – especifica a ordem de saída

A ordem das cláusulas na instrução SELECT não pode ser alterada. Apenas as duas primeiras cláusulas são obrigatórias: SELECT e FROM; as restantes são opcionais.

A Linguagem SQL

Caso de Estudo — A Base de Dados (simplificada)

Clientes

nr_cliente	nome	email	cidade
1000	Rita	rita@exemplo.pt	Braga
2000	Ana	ana@exemplo.pt	Guimarães
3000	Pedro	pedro@exemplo.pt	Vizela

Encomendas

nr_encomenda	nr_cliente	data	total	cidade
10	1000	20/01/2024	16.10	Braga
11	2000	11/02/2024	51.98	Guimarães
12	3000	07/07/2025	10.35	Guimarães
13	2000	12/12/2025	25.99	Fafe

Produtos

código	nome	preço	stock	categoria_id
1111	Rádio	10.35	10	EL
2222	Calças de ganga	25.99	20	RO
3333	Atum	1.15	30	CO
4444	iPad Air	350.10	5	EL

Categorias

id	nome	descrição
EL	Eletrónico	Inclui telemóveis e computadores.
RO	Roupa	Inclui peças de criança e adulto.
CO	Comida	Inclui todo o tipo de comida.

EncomendasProdutos

nr_encomenda	cod_produto	quantidade
10	1111	1
10	3333	5
11	2222	2
12	1111	1
13	2222	1

A Linguagem SQL

SELECT

Exemplos

Listar todos os detalhes de todos os clientes.

Como não há restrições nesta consulta, a cláusula WHERE é desnecessária e temos de apresentar todas as colunas.

```
SELECT nr_cliente, nome, email, cidade FROM clientes
```

utilizando um asterisco (*) no lugar dos nomes das colunas, é equivalente a indicar todas as colunas

```
SELECT * FROM clientes
```

A Linguagem SQL

SELECT

Exemplos

Listar todos apenas o nome de todos os clientes.

```
SELECT nome FROM clientes
```

nome
Rita
Ana
Pedro

Listar todos apenas o nome de todos clientes, atribuindo um nome temporário à coluna.

```
SELECT nome AS nome_cliente FROM clientes
```

nome_cliente
Rita
Ana
Pedro

A Linguagem SQL

SELECT

DISTINCT

Listar o número dos clientes que fizeram encomendas

```
SELECT nr_cliente FROM encomendas
```

nr_cliente
1000
2000
3000
2000

No resultado aparecem valores duplicados, ao contrário da operação Projeção da álgebra relacional o SELECT não elimina duplicados quando se projeta sobre uma ou mais colunas. Para eliminar os duplicados utilizamos o DISTINCT:

```
SELECT DISTINCT nr_cliente FROM encomendas
```

nr_cliente
1000
2000
3000

A Linguagem SQL

WHERE

Seleção de linhas (cláusula WHERE)

- Comparação – compara o valor de uma expressão com o valor de uma outra expressão.
- Conjunto – verifica se o valor de uma expressão cai dentro de um intervalo especificado de valores.
- Pertence – verifica se o valor de uma expressão pertence a um conjunto de valores.
- Pattern match – verifica se uma cadeia de caracteres corresponde a um padrão especificado.
- Null – testar se uma coluna tem um valor nulo (desconhecido).

A cláusula WHERE é equivalente à Seleção da álgebra relacional

A Linguagem SQL

WHERE

Comparação

Listar todas as encomendas com valor total superior a 50€.

```
SELECT * FROM encomendas WHERE total > 50x
```

nr_encomenda	nr_cliente	data	total	cidade
11	2000	11/02/2024	51.98	Guimarães

A Linguagem SQL

WHERE

Operadores disponíveis

= equals
<> diferente(padrão ISO)
< é inferior a
> é maior que

! = diferente (permitido em SGBD
<= é inferior ou igual a
> = é maior do que ou igual a

operadores lógicos AND, OR, e NOT, com parênteses (se necessário ou desejado) para alterar a precedência normal:

avaliar uma expressão da esquerda para a direita; entre parênteses são avaliadas em primeiro lugar; NOTs são avaliadas antes operadores AND e OR; ANDs são avaliadas antes dos ORs.

A Linguagem SQL

WHERE

Exemplos

Listar os números de todas as encomendas feitas em Guimarães ou Braga.

```
SELECT nr_encomenda FROM encomendas WHERE cidade = 'Guimarães' OR cidade = 'Braga'
```

nr_encomenda
10
11
12

Listar os produtos da categoria 'EL' cujo stock é maior do que 8.

```
SELECT * FROM produtos WHERE stock > 8 AND categoria = 'EL'
```

código	nome	preço	stock	categoria_id
1111	Rádio	10.35	10	EL

A Linguagem SQL

Condição de pesquisa

Intervalo (BETWEEN / NOT BETWEEN)

Listar as encomendas feitas entre 02/01/2024 e 27/03/2025 .

```
SELECT * FROM encomendas WHERE data BETWEEN '2024-01-02' AND '2024-03-27'
```

nr_encomenda	nr_cliente	data	total	cidade
10	1000	20/01/2024	16.10	Braga
11	2000	11/02/2024	51.98	Guimarães

```
SELECT * FROM encomendas WHERE data >= '2024-01-02' AND data <= '2024-03-27'
```

A Linguagem SQL

Condição de pesquisa

Pertence (IN / NOT IN)

Listar os clientes de Braga e os clientes de Vizela.

```
SELECT * FROM clientes WHERE cidade IN ('Braga','Vizela')
```

nr_cliente	nome	email	cidade
1000	Rita	rita@exemplo.pt	Braga
3000	Pedro	pedro@exemplo.pt	Vizela

```
SELECT * FROM clientes WHERE cidade = 'Braga' OR  cidade = 'Vizela'
```

A Linguagem SQL

Condição de pesquisa

Correspondencia a um padrão (LIKE / NOT LIKE)

O SQL tem dois símbolos especiais de correspondência de padrões:

% – representa qualquer sequência de zero ou mais caracteres (wildcard).
_ – representa qualquer caracter único.

Por exemplo:
LIKE 'H%' – o primeiro caracter deve ser H, mas o resto da cadeia pode ser vazia.
LIKE 'H_ _ _' – significa que deve haver exatamente quatro caracteres na sequência, a primeira das quais tem de ser um H.
LIKE '%e' – qualquer sequência de caracteres, de comprimento pelo menos 1, a última letra um e.

endereço LIKE '% Glasgow%', uma sequência de caracteres de qualquer comprimento contendo Glasgow.
endereço NOT LIKE 'H%', o primeiro caracter não pode ser um H.

A Linguagem SQL

Condição de pesquisa

Correspondencia a um padrão (LIKE / NOT LIKE)

Exemplo

Todos os clientes cujo o email não tem @.

```
SELECT * FROM clientes WHERE email NOT LIKE '%@%';
```

A Linguagem SQL

Condição de pesquisa

NULL (IS NULL / IS NOT NULL)

Exemplo

Todos os clientes que não têm email.

```
SELECT * FROM clientes WHERE email IS NULL;
```

Todos os clientes que têm email.

```
SELECT * FROM clientes WHERE email IS NOT NULL;
```

A Linguagem SQL

Ordenar Resultados

ORDER BY

A cláusula ORDER BY permite que as linhas sejam apresentadas por ordem ascendente (ASC) ou decrescente (DESC) de qualquer coluna ou combinação de colunas, independentemente do que aparece no resultado.

Em algumas implementações o ORDER BY só permite elementos da lista SELECT.

Em ambos os casos, a ORDER BY deve ser sempre a última cláusula da instrução SELECT.

A Linguagem SQL

Ordenar Resultados

ORDER BY

Exemplo

Listar os produtos cujo o preço está entre 5 e 30€, apresentando o resultado ordenado pelo stock (do maior para o menor).

```
SELECT * FROM produtos WHERE preço BETWEEN 5 AND 30 ORDER BY stock DESC;
```

código	nome	preço	stock	categoria_id
2222	Calças de ganga	25.99	20	RO
1111	Rádio	10.35	10	EL

A Linguagem SQL

Limitar resultados

LIMIT

Exemplo

Listar os produto com maior stock cujo o preço está entre 5 e 30€.

```
SELECT * FROM produtos WHERE preço BETWEEN 5 AND 30 ORDER BY stock DESC LIMIT 1;
```

código	nome	preço	stock	categoria_id
2222	Calças de ganga	25.99	20	RO
1111	Rádio	10.35	10	EL

A Linguagem SQL

SELECT

Utilização de Funções

- Visualização de um número:

```
SELECT 1;
```

- Fazendo o cálculo de uma expressão numérica:

```
SELECT 1 + (2 * 3) / 4;
```

- Visualizando uma string:

```
SELECT 'Olá Mundo!';
```

- Visualização da data e hora atuais do sistema:

```
SELECT NOW();
```

- Visualização do valor da constante matemática pi:

```
SELECT PI();
```

- Visualização dos valores da aplicação de funções trignométricas:

```
SELECT SIN(100), COS(90), TAN(PI());
```

- Conversão de valores em graus e radianos:

```
SELECT RADIANS(120); SELECT COS(RADIANS(90)); SELECT DEGREES(90);
```

A Linguagem SQL

SELECT

Datas e Horas

- Conversão da data atual nos seus respetivos dia da semana, dia, mês, ano e semana:

```
SELECT DAYNAME(NOW()), DAY(NOW()), MONTH(NOW()), YEAR(NOW()), WEEK(NOW());
```

- Obtenção da hora atual do sistema:

```
SELECT CURTIME();
```

- Obtenção da data atual do sistema:

```
SELECT CURDATE();
```

- Realização de uma operação de diferença entre dois elementos de dados do tipo DATETIME:

```
SELECT NOW(), Return_Date, DATEDIFF(NOW(), Return_Date) FROM Rental;
```

A Linguagem SQL

SELECT

Strings

- Concatenação de duas strings, com utilização de um separador:

```
SELECT customer_id, first_name, last_name,
CONCAT_WS(' ',first_name,last_name) AS "Nome Completo"
FROM customer ORDER BY CONCAT_WS(' ',first_name,last_name);
```

- Conversão para letras maiúsculas:

```
SELECT film_id, UPPER(title), UPPER(description) FROM film;
```

- Obter uma parte de uma string (substring):

```
SELECT SUBSTR(description, 1, 10) FROM film;
```

A Linguagem SQL

SELECT

Expressões regulares

As expressões regulares permitem pesquisar, filtrar e manipular dados textuais com base em padrões.

Operadores e funções

REGEXP ou RLIKE → verifica se um padrão existe
NOT REGEXP → verifica se NÃO existe
REGEXP_REPLACE() → substitui padrões (MySQL 8+)
REGEXP_SUBSTR() → extrai parte da string
REGEXP_INSTR() → retorna posição do padrão

A Linguagem SQL

SELECT

Expressões regulares

Exemplos

Encontrar clientes com nomes que começam com "A"

```
SELECT * FROM clientes
WHERE nome REGEXP '^A';
```

Emails válidos (simplificado)

```
SELECT * FROM clientes
WHERE email REGEXP '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$';
```

Substituir números numa string

```
SELECT REGEXP_REPLACE('abc123', '[0-9]', '');
```

A Linguagem SQL

Funções de agregação

A norma ISO define cinco funções agregadas:

- **COUNT** – retorna o número de valores de uma coluna especificada;
- **SUM** – retorna a soma dos valores de uma coluna especificada;
- **AVG** – retorna a média dos valores de uma coluna especificada;
- **MIN** – retorna o menor valor de uma coluna especificada;
- **MAX** – retorna o maior valor de uma coluna especificada.

Estas funções operaram numa única coluna de uma tabela e retornam um único valor. **COUNT, MIN, MAX** e aplicam-se a colunas numéricas e não numéricas, **o SUM e AVG** apenas podem ser utilizados em campos numéricos. Cada função elimina nulos primeiro e opera apenas sobre os valores não nulos restantes. **COUNT (*)** tem uma utilização especial, que conta todas as linhas de uma tabela.

A Linguagem SQL

Funções de agregação

COUNT(*)

Quantas encomendas foram feitas em 2024?

```
SELECT COUNT(*) as total_2024 FROM encomendas WHERE YEAR(data) = 2024
```

total_2024
2

COUNT(DISTINCT)

Quantas clientes diferentes fizeram encomendas com valor superior a 12€?

```
SELECT COUNT( DISTINCT nr_cliente) as nr_clientes FROM encomendas
WHERE valor > 12
```

nr_clientes
2

A Linguagem SQL

Funções de agregação

COUNT e SUM

Apresente o nr de produtos e a soma dos seus stocks.

```
SELECT COUNT(*) as total_prods, SUM(stock) AS total_stock
FROM produtos
```

total_prods	total_stock
4	65

A Linguagem SQL

Funções de agregação

MIN, MAX e AVG

Apresente o mínimo, máximo e média de preço dos produtos.

```
SELECT MIN(preço) AS prod_min,
MAX(preço) AS prod_max,
AVG(preço) as prod_avg
FROM produtos
```

prod_min	prod_max	prod_avg
1.15	350.10	96.90

A Linguagem SQL

Agrupar resultados

Cláusula GROUP BY

As colunas referenciadas no GROUP BY são designadas colunas de agrupamento. Quando GROUP BY é utilizado, cada item da lista de seleção deve ser de valor único por grupo. Além disso, a cláusula SELECT só pode conter:

- nomes de coluna;
- funções de agregação;
- constantes;
- uma expressão envolvendo combinações dos anteriores.

A Linguagem SQL

Agrupar resultados

Cláusula GROUP BY

Quantas encomendas fez cada cliente e qual a soma do valor gasto por cada um?

```
SELECT nr_cliente, COUNT(nr_encomenda) as contagem_encomendas, SUM(total) AS valor_total FROM encomendas GROUP BY nr_cliente
```

nr_encomenda	nr_cliente	data	total	cidade
10	1000	20/01/2024	16.10	Braga
11	2000	11/02/2024	51.98	Guimarães
12	3000	07/07/2025	10.35	Guimarães
13	2000	12/12/2025	25.99	Fafe



nr_cliente	contagem_encomendas	valor_total
1000	1	16.10
2000	2	77.97
3000	1	10.35

A Linguagem SQL

Agrupar resultados

Restrições em agrupamentos (cláusula HAVING)

Para cada cliente com mais de uma encomenda, mostra o valor médio que foi gasto.

```
SELECT nr_cliente, COUNT(nr_encomenda) as contagem, AVG(total) AS avg_total
FROM encomendas
GROUP BY nr_cliente
HAVING COUNT(nr_encomenda) > 1
ORDER BY nr_cliente;
```

nr_encomenda	nr_cliente	data	total	cidade
10	1000	20/01/2024	16.10	Braga
11	2000	11/02/2024	51.98	Guimarães
12	3000	07/07/2025	10.35	Guimarães
13	2000	12/12/2025	25.99	Fafe

nr_cliente	contagem	avg_total
2000	2	38.99

A Linguagem SQL

Subqueries

Uma instrução SELECT pode ser usada dentro outra instrução SELECT. Os resultados dessa instrução SELECT interna (ou subselect) são utilizados na instrução externa para ajudar a determinar o conteúdo do resultado final. Um subselect pode ser usado no WHERE e HAVING. Neste caso é chamado de uma subquery ou query aninhado. Subselects podem ser utilizados no INSERT,UPDATE, e DELETE.

Existem três tipos de subquerys:

- Um subquery escalar retorna uma única coluna, um único valor. Pode ser usado sempre que um valor único é necessário.
- Um subquery de linha retorna várias colunas, apenas uma única linha. Um subquery de linha pode ser usado sempre que é necessário um construtor de linha.
- Um subquery tabela retorna uma ou mais colunas e várias linhas. Um subquery tabela pode ser utilizado sempre que é necessário uma tabela, por exemplo, para o predicado IN.

A Linguagem SQL

Subqueries

Exemplo

Apresentar os produtos com preço acima da média e mostrar a diferença

```
SELECT
    codigo,
    nome,
    preco,
    preco - (SELECT AVG(preco) FROM Produtos) AS diferenca_media
FROM Produtos
WHERE preco > (SELECT AVG(preco) FROM Produtos);
```

média ≈ **96.90**

codigo	nome	preco	diferenca_media
4444	iPad Air	350.10	253.20

A Linguagem SQL

Subqueries

Subqueries aninhados (IN)

Apresente os produtos que pertencem a categorias do tipo "Eletrónico".

```
SELECT
    codigo,
    nome,
    preco
FROM Produtos
WHERE categoria_id IN (
    SELECT id
    FROM Categorias
    WHERE nome = 'Eletrónico'
);
```

Query EXTERNA

Query INTERNA → devolve os IDs das categorias eletrónicas (neste caso: EL)

codigo	nome	preco
1111	Rádio	10.35
4444	iPad Air	350.10

A Linguagem SQL

Subqueries

Subqueries aninhados (IN)

Atualizar o stock dos produtos para 0 que nunca foram encomendados

```
UPDATE Produtos
SET stock = 0
WHERE codigo IN (
    SELECT codigo
    FROM Produtos
    WHERE codigo NOT IN (
        SELECT cod_produto
        FROM EncomendasProdutos
    )
);
```

produtos que nunca foram encomendados

produtos que já foram encomendados

A Linguagem SQL

Combinação de Dados

- Os dados numa base de dados estão **distribuídos por várias tabelas**
- Para obter informação completa, usamos a instrução **JOIN**
- Os JOINs combinam tabelas através de **campos relacionados**
- Existem vários tipos de junção, com diferentes propósitos e formas de expressão. Todavia, em termos de SQL, podemos organizá-las em dois tipos:
  - Internas – INNER JOIN.
  - Externas – OUTER JOIN (LEFT, RIGHT e FULL)

A Linguagem SQL

Combinação de Dados – INNER JOIN

As operações de combinação de dados mais comuns são feitas através de **junções internas (INNER JOIN)**, que envolvem pelo menos duas tabelas ligadas por uma condição de junção.

As junções internas são também conhecidas como **equijunções**, pois a ligação entre as tabelas baseia-se numa **igualdade entre atributos**.

As operações de junção (JOIN) são muito importantes em sistemas de bases de dados porque:

- Permitem combinar dados de várias tabelas, produzindo um único conjunto de resultados
- Tendem a apresentar melhor desempenho do que operações com subqueries
- Devolvem apenas os registos que satisfazem a condição de junção definida

A Linguagem SQL

Combinação de Dados – INNER JOIN

Exemplos

Produtos com o nome da categoria

```
SELECT P.codigo, P.nome, P.preco, C.nome AS categoria
FROM Produtos AS P
INNER JOIN Categorias AS C
ON P.categoria_id = C.id;
```

Produtos de categorias específicas (ex: Eletrónico e Roupa)

```
SELECT P.nome, C.nome AS categoria
FROM Produtos AS P
INNER JOIN Categorias AS C
ON P.categoria_id = C.id
WHERE C.id IN ('EL', 'RO');
```

A Linguagem SQL

Combinação de Dados – INNER JOIN

Exemplos

Produtos encomendados com quantidade superior a 2

```
SELECT E.nr_encomenda, P.nome, EP.quantidade
FROM Encomendas AS E
INNER JOIN EncomendasProdutos AS EP
ON E.nr_encomenda = EP.nr_encomenda
INNER JOIN Produtos AS P
ON EP.cod_produto = P.codigo
WHERE EP.quantidade > 2;
```

Total de produtos por categoria

```
SELECT C.nome, COUNT(*) AS total_produtos
FROM Produtos AS P
INNER JOIN Categorias AS C
ON P.categoria_id = C.id
GROUP BY C.nome;
```

A Linguagem SQL

Combinação de Dados – INNER JOIN

Exemplos

Encomendas de clientes específicos com produtos eletrónicos

```
SELECT CL.nome, P.nome AS produto
FROM Clientes AS CL
INNER JOIN Encomendas AS E
ON CL.nr_cliente = E.nr_cliente
INNER JOIN EncomendasProdutos AS EP
ON E.nr_encomenda = EP.nr_encomenda
INNER JOIN Produtos AS P
ON EP.cod_produto = P.codigo
WHERE CL.nr_cliente IN (1000,2000)
AND P.categoria_id = 'EL';
```

A Linguagem SQL

Combinação de Dados – INNER JOIN

Exemplos

Total de produtos encomendados por produto

```
SELECT
    P.nome,
    SUM(EP.quantidade) AS total_vendido
FROM Produtos AS P
INNER JOIN EncomendasProdutos AS EP
ON P.codigo = EP.cod_produto
GROUP BY P.nome;
```

Produtos com mais de 5 unidades vendidas

```
SELECT
P.nome,
SUM(EP.quantidade) AS total_vendido
FROM Produtos AS P
INNER JOIN EncomendasProdutos AS EP
ON P.codigo = EP.cod_produto
GROUP BY P.nome
HAVING SUM(EP.quantidade) > 5;
```

A Linguagem SQL

Combinação de Dados – INNER JOIN

Exemplos

Total de produtos encomendados por produto

```
SELECT
    P.nome,
    SUM(EP.quantidade) AS total_vendido
FROM Produtos AS P
INNER JOIN EncomendasProdutos AS EP
ON P.codigo = EP.cod_produto
GROUP BY P.nome;
```

Produtos com mais de 5 unidades vendidas

```
SELECT
P.nome,
SUM(EP.quantidade) AS total_vendido
FROM Produtos AS P
INNER JOIN EncomendasProdutos AS EP
ON P.codigo = EP.cod_produto
GROUP BY P.nome
HAVING SUM(EP.quantidade) > 5;
```

A Linguagem SQL

Combinação de Dados – OUTER JOIN

As junções externas, tal como as junções internas, permitem **combinar dados de duas ou mais tabelas** com base numa condição de junção.

No entanto, diferenciam-se por incluírem também nos resultados **registos que não têm correspondência** na outra tabela. Dependendo do tipo de junção, são considerados todos os registos de uma das tabelas ou de ambas.

As junções externas podem ser:

LEFT OUTER JOIN → inclui todos os registos da tabela da esquerda
RIGHT OUTER JOIN → inclui todos os registos da tabela da direita
FULL OUTER JOIN → inclui todos os registos de ambas as tabelas

Ou seja, permitem identificar **dados em falta ou sem correspondência**.

A Linguagem SQL

Combinação de Dados – OUTER JOIN

LEFT OUTER JOIN

Exemplo

Listar todos os produtos + encomendas (se existirem)

```
SELECT
    P.codigo,
    P.nome,
    EP.nr_encomenda
FROM Produtos AS P
LEFT JOIN EncomendasProdutos AS EP
ON P.codigo = EP.cod_produto;
```

codigo	nome	nr_encomenda
1111	Rádio	1
2222	Calças de ganga	1
3333	Atum	2
4444	iPad Air	NULL

- Mostra todos os produtos
- Se o produto foi encomendado → aparece o nº da encomenda
- Se não foi → aparece NULL

A Linguagem SQL

Combinação de Dados – OUTER JOIN

RIGT OUTER JOIN

Exemplo

Listar todas as encomendas, indicando o nome do cliente da mesma cidade, caso exista.

```
SELECT CL.nome, CL.cidade, E.nr_encomenda
FROM Clientes CL
RIGHT JOIN Encomendas E
ON CL.cidade = E.cidade;
```

nome	cidade	nr_encomenda
Rita	Braga	10
Ana	Guimarães	11
Ana	Guimarães	12
NULL	NULL	13

- RIGHT JOIN → queremos todas as encomendas
- Algumas podem não ter cliente na mesma cidade
- Mesmo assim → aparecem (com NULL)

A Linguagem SQL

Combinação de Dados – OUTER JOIN

FULL OUTER JOIN

Exemplo

Apresentar todos os clientes e todas as encomendas da mesma cidade, mesmo que não exista correspondência.

```
SELECT CL.nome, CL.cidade,
     E.nr_encomenda, E.cidade
FROM Clientes CL
FULL OUTER JOIN Encomendas E
ON CL.cidade = E.cidade;
```

⚠ MySQL não suporta `FULL OUTER JOIN`

```
SELECT CL.nome, CL.cidade, E.nr_encomenda, E.cidade
FROM Clientes CL
LEFT JOIN Encomendas E
ON CL.cidade = E.cidade

UNION

SELECT CL.nome, CL.cidade, E.nr_encomenda, E.cidade
FROM Clientes CL
RIGHT JOIN Encomendas E
ON CL.cidade = E.cidade;
```

A Linguagem SQL

Combinação de Dados – OUTER JOIN

FULL OUTER JOIN

Exemplo

Apresentar todos os clientes e todas as encomendas da mesma cidade, mesmo que não exista correspondência.

```
SELECT CL.nome, CL.cidade,
     E.nr_encomenda, E.cidade
FROM Clientes CL
FULL OUTER JOIN Encomendas E
ON CL.cidade = E.cidade;
```

⚠ MySQL não suporta `FULL OUTER JOIN`

```
SELECT CL.nome, CL.cidade, E.nr_encomenda, E.cidade
FROM Clientes CL
LEFT JOIN Encomendas E
ON CL.cidade = E.cidade

UNION

SELECT CL.nome, CL.cidade, E.nr_encomenda, E.cidade
FROM Clientes CL
RIGHT JOIN Encomendas E
ON CL.cidade = E.cidade;
```

A Linguagem SQL

Combinação de Dados – SEMI JOIN

A operação de **semi-junção (semi join)** permite reduzir o número de registos envolvidos numa junção entre duas tabelas, devolvendo apenas os atributos de uma delas.

Numa **semi-junção à esquerda**, são apresentados apenas os atributos da primeira tabela; numa **semi-junção à direita**, apenas os da segunda. Em ambos os casos, apenas são considerados os registos que têm correspondência na outra tabela.

Na prática, é uma operação bastante comum, normalmente implementada através de **EXISTS** ou **IN**, sendo útil para verificar a existência de relações sem necessidade de devolver todos os dados da junção.

A Linguagem SQL

Manipulação de Dados

EXISTS e NOT EXISTS

Apenas podem ser usadas em subquerys e produzem um resultado verdadeiro ou falso.

EXISTS é verdadeira se, e só se existir pelo menos uma linha na tabela retornada pelo subquery; é falsa se o subquery retornar vazio.

NOT EXISTS é o oposto do EXISTS.

EXISTS e NOT EXISTS verificam apenas a existência ou não existência de linhas no resultado do subquery. A subquery pode conter qualquer número de colunas.

Para simplificar, usar a seguinte forma: (SELECT * FROM…)

A Linguagem SQL

Manipulação de Dados

EXISTS e NOT EXISTS

Exemplos

Clientes que já fizeram encomendas

```
SELECT CL.*
FROM Clientes CL
WHERE EXISTS (
    SELECT *
    FROM Encomendas E
    WHERE E.nr_cliente = CL.nr_cliente
);
```

Apresentar os produtos que nunca foram encomendados

```
SELECT P.codigo, P.nome
FROM Produtos P
WHERE NOT EXISTS (
    SELECT 1
    FROM EncomendasProdutos EP
    WHERE EP.cod_produto = P.codigo
);
```

A Linguagem SQL

Combinar Tabelas

UNION, INTERSECT, EXCEPT

Em SQL, podemos usar as operações de **União, Interseção e Diferença** para combinar os resultados de dois ou mais querys num único resultado:

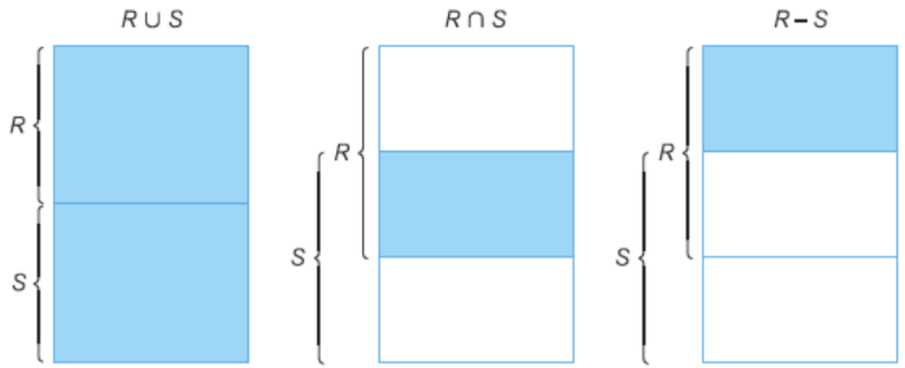
- A União de duas tabelas, A e B, é uma tabela com todas as linhas que estão, na primeira tabela A ou na segunda tabela B ou em ambas.
- A Interseção de duas tabelas, A e B, é uma tabela com todas as linhas que são comuns às duas tabelas A e B.
- A Diferença de duas tabelas, A e B, é uma tabela com todas as linhas que estão na tabela A mas não estão na tabela B.

A Linguagem SQL

Combinar Tabelas

UNION, INTERSECT, EXCEPT

Isto implica que as duas tabelas deve conter o mesmo número de colunas, e que as colunas possuem os mesmos tipos de dados e comprimentos.



operator [ALL] - Se ALL for utilizado, o resultado pode incluir linhas duplicadas.

A Linguagem SQL

Combinar Tabelas

UNION

Exemplo

Cidades de clientes de Braga + cidades de encomendas recentes

```
SELECT cidade
FROM Clientes
WHERE cidade = 'Braga'

UNION

SELECT cidade
FROM Encomendas
WHERE data > '2024-01-01';
```

A Linguagem SQL

Combinar Tabelas

INTERSECT

Exemplo

Produtos que existem na tabela Produtos e também foram encomendados

```
SELECT codigo FROM Produtos
INTERSECT
SELECT cod_produto FROM EncomendasProdutos;
```

A Linguagem SQL

Combinar Tabelas

Diferença (EXCEPTS)

Exemplo

Apresentar as cidades que existem em clientes mas não em encomendas

```
SELECT cidade FROM Clientes
EXCEPT
SELECT cidade FROM Encomendas;
```

Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

A Liguagem SQL (continuação)



António Abelha | abelha@di.uminho.pt